MINISTRY OF EDUCATION AND TRAINING
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING
FACULTY OF ADVANCED EDUCATION

COURSE REPORT

SEMESTER 1, 2026-2027

Summary of content

Advisor: Huynh Hoang Ha, M.Eng.

Implementation Group: 7

1. Nguyen Phu Huu – 24161090
2. Nguyen Trung Nguyen – 24161115
3. Trinh Ngoc Tin – 24161140

Ho Chi Minh City, October 2026

CHAPTER 1: LINUX

1. Embedded Linux Architecture & Concurrent Processing Mechanism

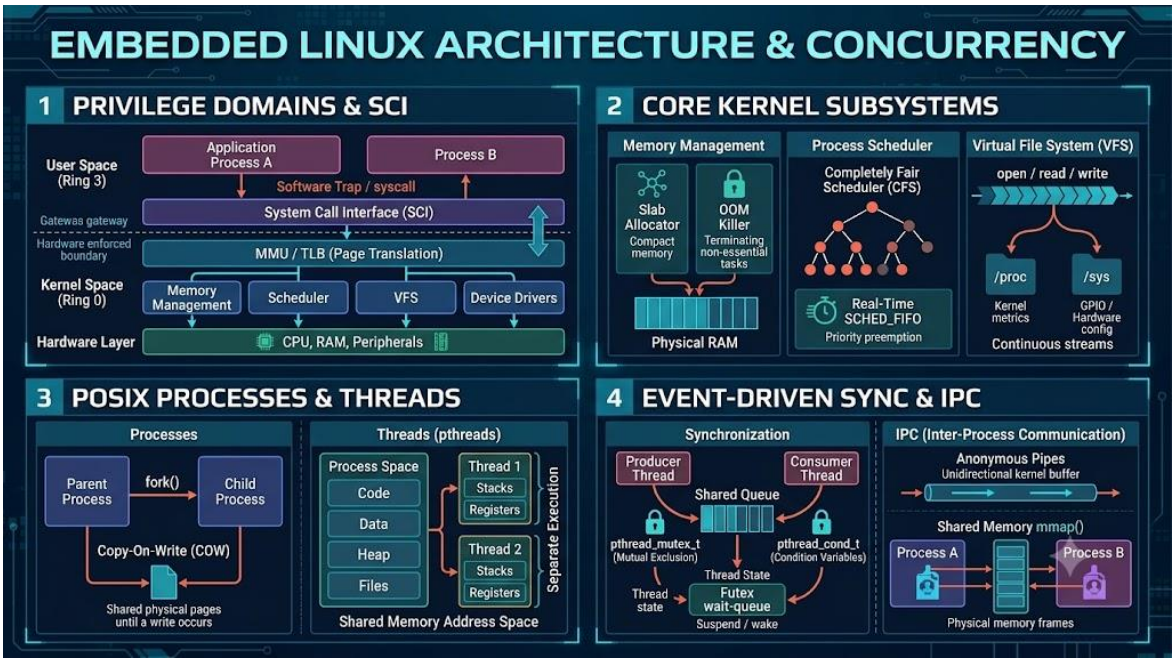


Figure 1.1. Embedded Linux architecture, kernel services, and concurrency mechanisms.

Privilege domains and memory isolation. Linux separates user processes from kernel services through hardware privilege modes and virtual memory. The MMU and TLB translate addresses and isolate each process. Invalid user-space memory access may trigger a segmentation fault (SIGSEGV), whereas a severe kernel fault may cause a kernel panic.

System Call Interface (SCI). User applications request protected operating-system services through system calls. Library wrappers pass system-call numbers and arguments to the CPU; a trap switches execution into the kernel, which validates the request, performs the operation and returns the result to user space.

Core Linux subsystems

Memory management. The kernel allocates and reclaims memory through physical-page management and slab-family allocators; the out-of-memory (OOM) killer can reclaim resources when RAM is exhausted.

Process scheduling. The Completely Fair Scheduler (CFS) shares CPU time among tasks, while SCHED_FIFO and SCHED_RR support real-time priorities for time-critical threads.

Virtual File System (VFS). The VFS standardizes file and device access through APIs such as open(), read(), write() and close(); /proc and /sys expose operating-system and device information.

Processes and POSIX threads. Processes have independent virtual address spaces, and fork() commonly uses copy-on-write to reduce duplication costs. POSIX threads share process memory and open files while retaining separate stacks and execution contexts.

Synchronization and IPC. Mutexes protect shared data, and condition variables let consumer threads wait without busy polling. Pipes and shared memory support communication across processes. Tools such as /proc, htop and ThreadSanitizer help diagnose resource use and data races.

2. Hardware-Software Co-Design & Microarchitectural Optimization

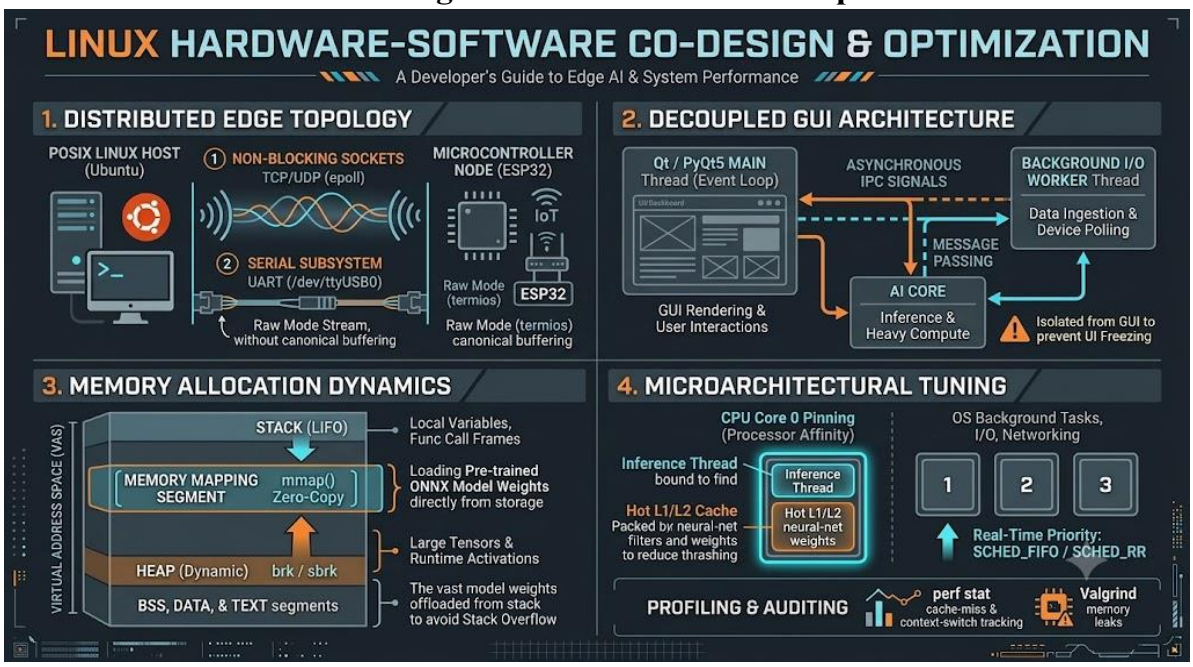


Figure 1.2. Linux hardware–software co-design and microarchitectural optimization.

Distributed edge architecture. The Ubuntu/Linux host handles image inference, multithreading and visualization, while the ESP32 provides button-driven inputs and LED feedback. This partition keeps interactive control near hardware and compute-intensive work on the host.

Non-blocking communication and responsive GUI

POSIX sockets. Non-blocking file descriptors (O_NONBLOCK) with select(), poll() or epoll() allow I/O threads to process events without blocking the GUI. TCP serves reliable commands; UDP may serve low-latency telemetry.

UART using termios. The Linux host communicates through a serial device such as /dev/ttyUSB0. Raw mode disables canonical input and echo processing, preserving byte-oriented binary messages.

Event-driven GUI. PyQt5 keeps serial and network I/O in workers or child processes; signals and IPC deliver results back to the GUI, preventing the main event loop from freezing.

CNN memory allocation. Small local values are kept on the stack; image tensors and model data use heap or mapped memory as appropriate. This avoids excessive stack use and reduces unnecessary copying.

Core affinity and profiling. CPU affinity can stabilize inference measurements by reducing thread migration. Tools such as perf, pidstat and Valgrind support performance and memory diagnostics.

3. Distributed Edge AI Deployment on Linux – LeNet-5

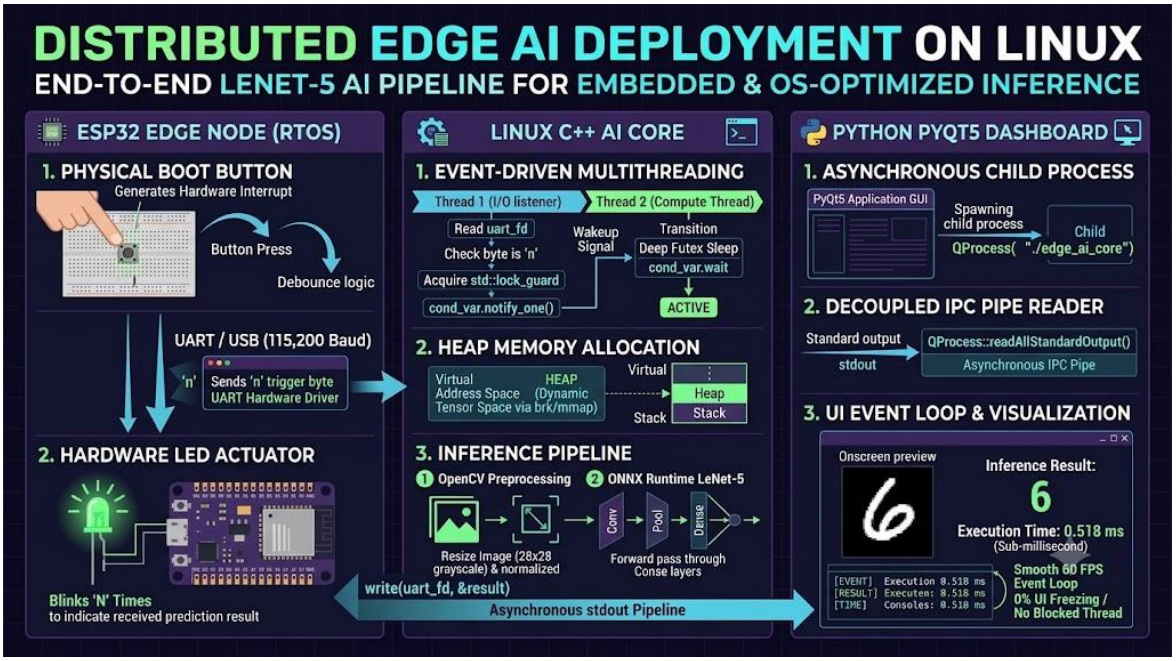


Figure 1.3. Distributed Linux–ESP32 edge-AI inference and feedback workflow.

ESP32 request and response. After its BOOT button is pressed, the ESP32 debounces the input and sends the character “n” to the Linux host over 115200-baud

UART. The host returns the predicted digit; the ESP32 blinks its LED that number of times.

Event-driven C++ inference core (edge_ai_core)

UART listener thread. The I/O thread watches the serial port and, on receiving “n”, sets trigger_ai under a mutex before calling cond_var.notify_one().

Inference thread. The compute thread waits on the condition variable until triggered. It then prepares a grayscale 28 × 28 image with OpenCV and invokes the LeNet-5 model through ONNX Runtime.

PyQt5 monitoring and reported results

GUI integration. PyQt5 starts the C++ backend with QProcess and reads tagged result and timing messages asynchronously from standard output.

Observed behavior. The dashboard remained responsive in the reported demonstration, and the source document recorded inference times of 0.518–4.223 ms. These measurements describe the demonstrated host setup, not the FPGA CNN accelerator.

CHAPTER 2: RISC-V

1. RISC-V architecture overview

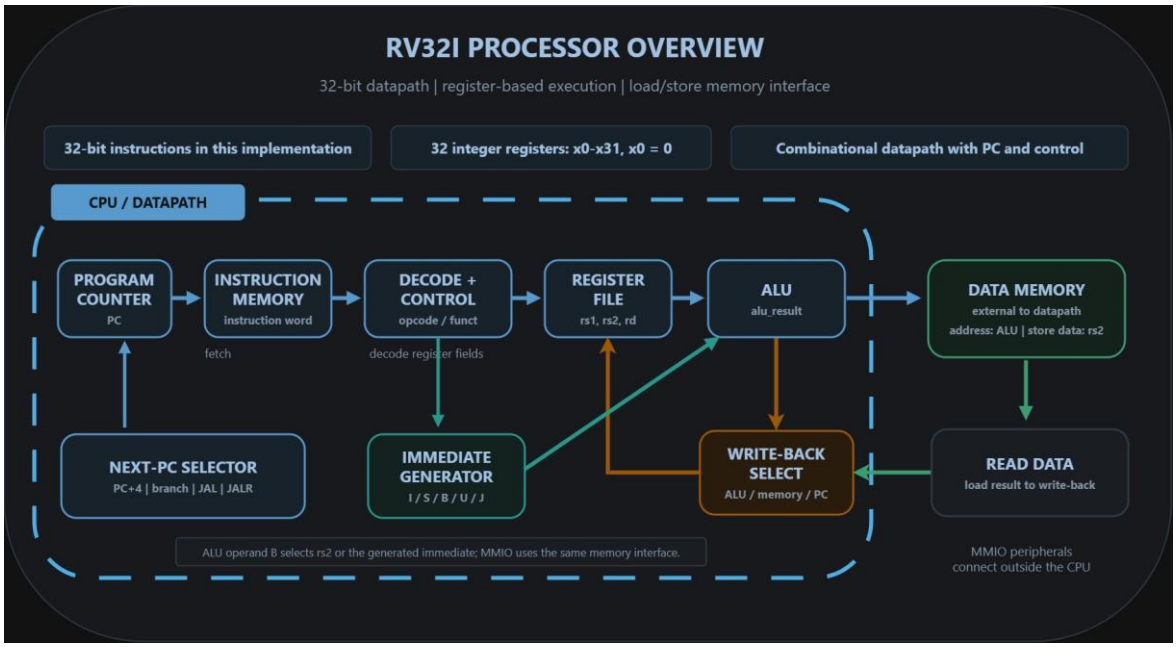


Figure 2.1. RV32I processor datapath and instruction execution components.

The core uses a 32-bit datapath with a program counter, instruction memory, control unit, register file, immediate generator, ALU, and write-back selection. The register file contains 32 integer registers; x0 reads as zero. Branch conditions compare the register operands, while JAL and JALR select jump targets. Data memory is connected through the CPU memory interface outside the datapath, allowing the surrounding SoC to route memory-mapped peripheral accesses.

2. Implementation and UART program loading

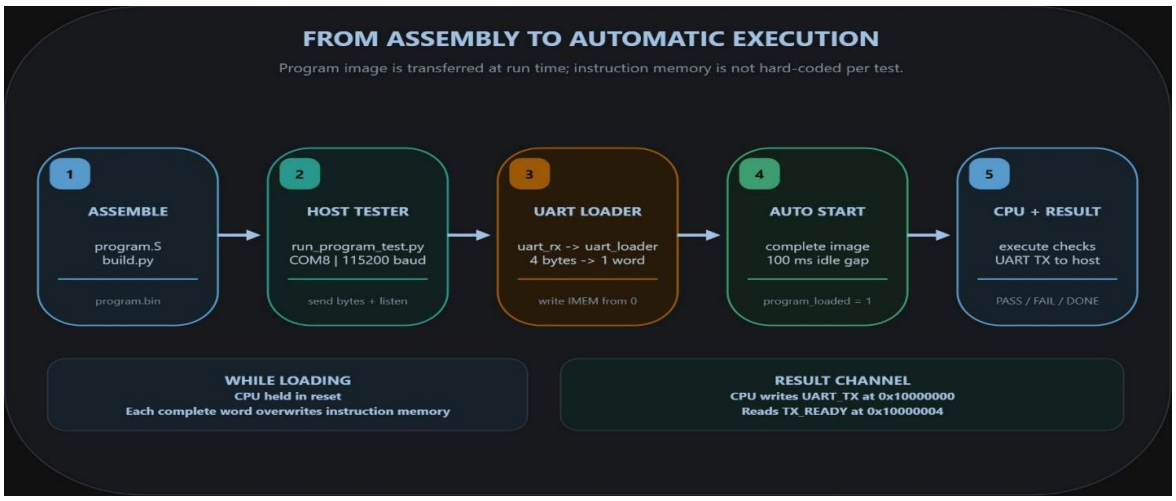


Figure 2.2. UART firmware loading and automatic CPU execution sequence.

The assembly test is compiled into a binary image, then the Python host runner sends that image over UART. The receiver and loader assemble the serial byte stream into 32-bit instruction words and write them sequentially into instruction memory. The CPU remains held in reset during transfer and starts automatically after the complete image is followed by a 100 ms idle interval. The same host script then collects the test status sent by the CPU through UART MMIO; no manual start-button action is needed.

3. Test coverage and observed result



Figure 2.3. FPGA RV32I instruction-test results for six test groups.

The board run passed the six programmed groups: arithmetic and logic, shift and comparison, data memory, branches, LUI/AUIPC, and JAL/JALR. The terminal capture reports completion and 6/6 passed. These results show successful operation for the selected instruction cases and test data; broader instruction coverage and corner-case verification remain future work.

CHAPTER 3: CNN ACCELERATOR

1. Overview and Hardware Architecture

The system implements a signed-INT8 CNN convolution accelerator as a 32-bit MMIO peripheral in a team-developed RV32I SoC. Conv1 and Conv2 are accelerated in RTL; pooling, requantization, fully connected layers and Argmax run in C firmware. The accelerator includes input and weight buffers, address generation, a data feeder, a 4 × 4 PE array, INT32 accumulators, hardware ReLU and an output buffer.

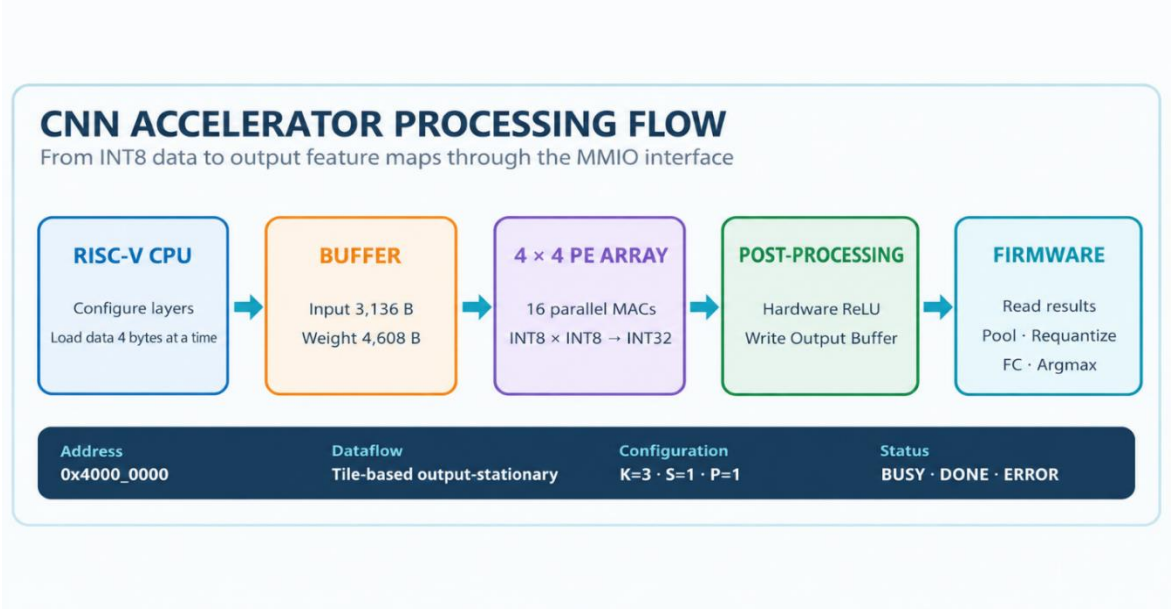


Figure 3.1. CNN accelerator processing flow and hardware–software partitioning.

Architecture and Key Characteristics

Four signed-INT8 activations and four weights are dispatched per compute step, enabling 16 parallel multiply–accumulate operations. Output-stationary tiling retains partial sums within the PEs until a tile completes, reducing repeated buffer traffic. The theoretical arithmetic peak is 86.4 MMAC/s at a 5.4 MHz SoC clock; actual throughput is reduced by data movement and control overhead.

Arithmetic	Parallelism	Theoretical peak
INT8 × INT8 → INT32	4 × 4 PE = 16 MAC/cycle	86.4 MMAC/s at 5.4 MHz

2. Datapath Operation and MMIO Control

Firmware clears state, configures tensor dimensions and convolution parameters (typically K = 3, S = 1, P = 1), then loads four INT8 values per 32-bit write. The controller validates the configuration and data counts. During computation, the

sequencer traverses output positions, channels and kernel taps, masking invalid lanes; INT32 results pass through ReLU into the output buffer.

Five-Stage Control Sequence

Step	Phase	Operation
1	CLEAR / CONFIG	Reset status and latch layer settings.
2	LOAD	Load packed INT8 activations/weights; count writes.
3	CHECK	Reject invalid settings, insufficient data or overflow.
4	COMPUTE	Iterate kernel and channel loops; mask invalid lanes.
5	DONE / READ	Hold DONE until readout and the next CLEAR.

Convolution Workload

Layer	Input	Weights	Output	MACs
Conv1	784 INT8	144 INT8	12,544 INT32	112,896
Conv2	3,136 INT8	4,608 INT8	6,272 INT32	903,168

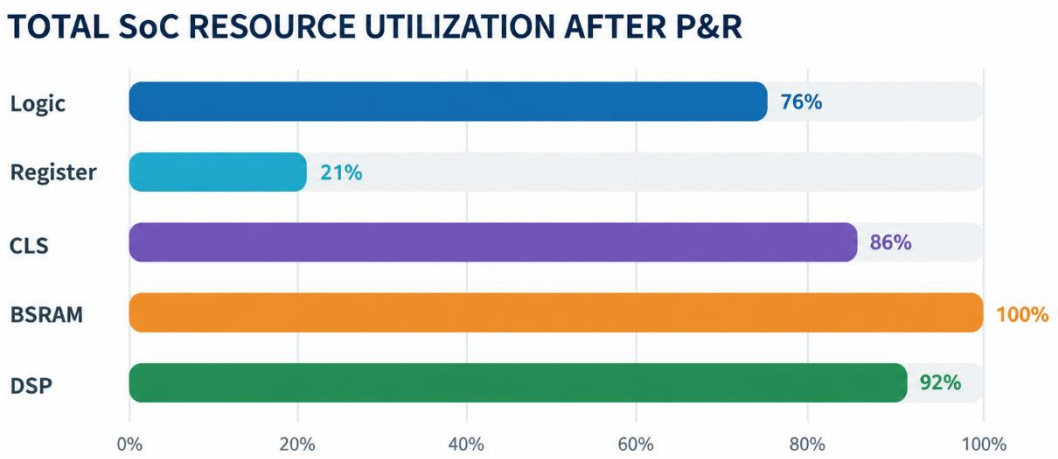
Convolutions total 1,016,064 MACs (83.4% of the network); Conv2 accounts for 74.2%.

Principal MMIO Registers — Base 0x4000_0000

Offset	Register(s)	Purpose
0x00 / 0x04	CTRL / STATUS	START, CLEAR; BUSY, DONE, ERROR.
0x08–0x24	CONFIG	H, W, Cin, Cout, K, S, P, ReLU.
0x30 / 0x34	INPUT / WEIGHT	Write four packed signed INT8 values.
0x38 / 0x44	OUTPUT / COUNT	Read INT32 result and output count.
0x48	ERROR_CODE	Diagnose configuration/data/capacity faults.

3. Verification, FPGA Results and Assessment

Verification combines RTL tensor comparisons with an INT32 golden model, firmware-C end-to-end simulation with an SDHC model, and synthesis/place-and-route of the complete SoC on GW2AR-18. The exported integer pipeline classified 98 of 100 images correctly; end-to-end regression checks prediction and confidence through



Result MMIO.

Figure 3.2. Total SoC resource utilization after place-and-route.

Post-P&R Resource Utilization

Resource	Reported use	Utilization
Logic	15,715 / 20,736	76%
Registers	3,264 / 15,750	21%
CLS	Not reported as a count	86%
BSRAM	46 / 46	100%
DSP	22 / 24	92%

Timing, Limitations and Conclusion

System, SoC and pixel clocks are 27, 5.4 and 27 MHz; reported Fmax values are 137.825, 6.617 and 33.803 MHz. Setup timing passes, but 15 hold endpoints remain, mostly across soc_clk–sys_clk. BSRAM (100%) and DSP (92%) constrain expansion. Priorities are CDC synchronization, repeated timing analysis and on-board latency/PE-utilization measurements. The RTL accelerator demonstrates INT8 convolution offload integrated with RISC-V, microSD and HDMI, pending timing closure.